

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №2
по дисциплине «Информационные технологии»
Тема: «Алгоритмы и структуры данных в Python.»

Студент гр. 5387

Степанов Леонид
Артемович

Преподаватель

Клюканов Виталий
Константинович

Санкт-Петербург
2026

СОДЕРЖАНИЕ

Цель работы	3
Изучение базовых структур данных и алгоритмов на языке Python.	
Приобретение практических навыков реализации связанного однонаправленного списка, управления памятью (ссылками) и анализа вычислительной сложности алгоритмов.	3
Задание.....	3
Реализовать связный однонаправленный список, состоящий из двух зависимых классов: Node (элемент списка) и LinkedList (сам список). Обеспечить методы для добавления, удаления, изменения элементов, а также строкового представления списка. В отчете дать теоретическую сводку об отличиях связанного списка от массива, указать сложность каждого реализованного метода и описать специфику бинарного поиска в связном списке.....	3
Основные теоретические положения.....	3
Выполнение работы.....	6
Выводы	7
ПРИЛОЖЕНИЕ А	8

Цель работы

Изучение базовых структур данных и алгоритмов на языке Python. Приобретение практических навыков реализации связного однонаправленного списка, управления памятью (ссылками) и анализа вычислительной сложности алгоритмов.

Задание

Реализовать связный однонаправленный список, состоящий из двух зависимых классов: Node (элемент списка) и LinkedList (сам список). Обеспечить методы для добавления, удаления, изменения элементов, а также строкового представления списка. В отчете дать теоретическую сводку об отличиях связного списка от массива, указать сложность каждого реализованного метода и описать специфику бинарного поиска в связном списке.

Основные теоретические положения

1. Понятие связного списка и отличия от массива Связный список — это базовая динамическая структура данных, состоящая из узлов, каждый из которых содержит сами данные и ссылку (указатель) на следующий узел в последовательности.

Основные отличия связного списка от массива (классического списка Python, реализованного как динамический массив):

- **Выделение памяти:** Элементы массива хранятся в памяти непрерывным блоком. Узлы связного списка разбросаны в памяти произвольно, связь поддерживается только за счет указателей.
- **Доступ к элементам:** В массиве возможен произвольный доступ к любому элементу по индексу за $O(1)$. В связном списке доступ строго последовательный — чтобы добраться до n -го элемента, нужно пройти от начала списка шагов, что занимает времени $O(n)$.
- **Вставка и удаление:** В массиве вставка или удаление в начале или середине требует сдвига всех последующих элементов, что занимает

времени. В связном списке, если указатель уже находится на нужном месте, вставка и удаление происходят за путем переопределения ссылок.

2. Вычислительная сложность реализованных методов

- **Класс Node:**

- `__init__` —
- `get_data` —
- `change_data` —
- `__str__` —

- **Класс LinkedList:**

- `__init__` —
- `__len__` —
- `append` — . В текущей реализации алгоритм проходит весь список от головы (`head`) до конца (пока `curr.next` не станет `None`), чтобы добавить новый элемент.
- `__str__` — , так как требуется обойти весь список для формирования строки.
- `pop` — . Для удаления последнего элемента необходимо найти предпоследний узел, что требует обхода всего списка.
- `change_on_start` — . Алгоритм проходит шагов от начала списка для поиска нужного элемента.
- `clear` — , так как достаточно обнулить ссылку на `head` и длину списка (сборщик мусора Python сам удалит недоступные узлы).

3. Бинарный поиск в связном списке Классический алгоритм бинарного поиска требует произвольного доступа к элементам коллекции для вычисления среднего элемента на каждом шаге (в массиве это делается за $O(1)$). В однонаправленном связном списке найти элемент посередине можно только последовательным перебором, что занимает времени на каждой итерации. Из-за этого общая временная сложность бинарного поиска в связном списке вырождается и становится хуже линейного поиска — $O(n)$. По этой причине

бинарный поиск для связного списка на практике не применяется; вместо него используют обычный линейный поиск за или другие структуры данных (например, бинарные деревья поиска). В стандартном списке Python (массиве) бинарный поиск работает эффективно за .

Выполнение работы

Класс Node Описывает узел связного списка.

- **Поля:** data (хранимое значение) и next (ссылка на следующий узел).
- **Методы:** *
 - **__init__:** Инициализирует узел переданными данными, по умолчанию next равно None.
 - **get_data и change_data:** Методы доступа и изменения поля data.
 - **__str__:** Возвращает отформатированную строку с информацией о текущих данных и данных следующего узла.

Класс LinkedList Описывает структуру данных «Связный список».

- **Поля:** head (ссылка на первый узел) и length (текущее количество узлов).
- **Методы:**
 - **__init__:** Конструктор. Создает пустой список или список из одного элемента.
 - **__len__:** Возвращает актуальную длину списка, хранящуюся в поле length.
 - **append:** Добавляет новый элемент в конец списка. Если список пуст, элемент становится головой. Иначе происходит итерация до последнего узла.
 - **__str__:** Проходит по всем узлам и формирует итоговую строку согласно заданному формату.
 - **pop:** Удаляет последний элемент списка. При попытке удаления из пустого списка генерирует исключение IndexError.
 - **change_on_start:** Изменяет данные -го элемента с начала списка. Если передан некорректный индекс, генерирует KeyError.
 - **clear:** Очищает список, сбрасывая состояние head в None и длину в 0.

Выводы

В результате выполнения лабораторной работы были закреплены знания о динамических структурах данных и освоены навыки реализации однонаправленного связного списка на языке Python. Были реализованы классы Node и LinkedList с основными методами: добавлением элемента (append), извлечением (pop), очисткой (clear) и изменением элемента (change_on_start). В ходе работы был проведен анализ асимптотической сложности реализованных методов. Также была изучена специфика работы алгоритма бинарного поиска: было установлено, что из-за отсутствия произвольного доступа к памяти за $O(n)$, бинарный поиск неэффективен для связных списков в отличие от массивов (классических списков Python).

ПРИЛОЖЕНИЕ А

Исходный код программы (Файл main.py)

```
class Node:

    def __init__(self, data, next=None):
        self.data = data
        self.next = next

    def get_data(self):
        return self.data

    def change_data(self, new_data):
        self.data = new_data

    def __str__(self):
        next_data = self.next.data if self.next else None
        return f"data: {self.data}, next: {next_data}"

class LinkedList:

    def __init__(self, head=None):
        self.head = head
        self.length = 1 if head else 0

    def __len__(self):
        return self.length

    def append(self, element):
        new_node = Node(element)
```



```

if self.head is None:
    self.head = new_node
else:
    curr = self.head
    while curr.next:
        curr = curr.next
    curr.next = new_node

self.length += 1

def __str__(self):
    if self.length == 0:
        return "LinkedList[]"

    result = []
    curr = self.head

    while curr:
        next_data = curr.next.data if curr.next else None
        result.append(f'data: {curr.data}, next: {next_data}')
        curr = curr.next

    return f'LinkedList[length = {self.length}, [{'; '.join(result)}]]'

def pop(self):
    if self.length == 0:
        raise IndexError("LinkedList is empty!")

    if self.length == 1:

```

```

        self.head = None
    else:
        curr = self.head
        while curr.next.next:
            curr = curr.next
        curr.next = None

    self.length -= 1

def change_on_start(self, n, new_data):
    if n <= 0 or n > self.length:
        raise KeyError("Element doesn't exist!")

    curr = self.head
    for _ in range(n - 1):
        curr = curr.next

    curr.data = new_data

def clear(self):
    self.head = None
    self.length = 0
    raise ValueError('Invalid value')

    self.author = author
    self.hardcover = hardcover
    self.pages = pages

def __str__(self):
    return (f'Book: название {self.name}, цена {self.price}, "

```

```
f"возрастное ограничение {self.age_limit}, стиль {self.style}, "  
f"автор {self.author}, твердый переплет {self.hardcover}, "  
f"количество страниц {self.pages}.")
```

```
def __eq__(self, other):  
    if isinstance(other, Book):  
        return self.name == other.name and self.author == other.author  
    return False
```

```
class Newspaper(Edition):
```

```
    def __init__(self, name: str, price: int, age_limit: int, style: str,  
                  online_edition: bool, country: str, frequency: int):  
        super().__init__(name, price, age_limit, style)
```

```
    if not isinstance(online_edition, bool):
```

```
        raise ValueError('Invalid value')
```

```
    if not isinstance(country, str) or country == "":
```

```
        raise ValueError('Invalid value')
```

```
    if not isinstance(frequency, int) or frequency <= 0:
```

```
        raise ValueError('Invalid value')
```

```
    self.online_edition = online_edition
```

```
    self.country = country
```

```
    self.frequency = frequency
```

```
def __str__(self):
```

```
    return (f"Newspaper: название {self.name}, цена {self.price}, "  
           f"возрастное ограничение {self.age_limit}, стиль {self.style}, "
```

```
f"интернет издание {self.online_edition}, страна {self.country}, "  
f"периодичность {self.frequency}."")
```

```
def __eq__(self, other):  
    if isinstance(other, Newspaper):  
        return self.name == other.name and self.country == other.country  
    return False
```

```
class BookList(list):  
    def __init__(self, name):  
        super().__init__()  
        self.name = name  
  
    def append(self, p_object):  
        if isinstance(p_object, Book):  
            super().append(p_object)  
        else:  
            raise TypeError(f"Invalid type {type(p_object)}")  
  
    def total_pages(self):  
        return sum(book.pages for book in self)  
  
    def print_count(self):  
        print(len(self))
```

```
class NewspaperList(list):  
    def __init__(self, name):
```

```
super().__init__()  
self.name = name
```

```
def extend(self, iterable):  
    if not hasattr(iterable, '__iter__'):  
        raise TypeError  
    for item in iterable:  
        if isinstance(item, Newspaper):  
            super().append(item)
```

```
def print_age(self):  
    if self:  
        print(min(n.age_limit for n in self))
```

```
def print_total_price(self):  
    print(sum(n.price for n in self))
```